

型付き AI エージェント記述言語の設計と実装

児玉靖司 (Yasushi Kodama)
法政大学 (Hosei University)

2026 年 5 月 12 日

概要

本論文では、生成 AI を外部サービスではなくプログラム構成要素として扱うための型付き AI エージェント記述言語 AIPL (Actor-based Intelligent Parallel Language) の設計と実装について述べる。AIPL はアクターを基本単位とし、非同期メッセージ送信、同期呼び出し、Future、動的な振る舞い変更、AI 呼び出しを同一の言語モデル上に統合する。さらに Python 版では、型注釈、Union 型、Generics、Capability 効果、CSP 風チャンネル、線形型、所有権、transient cast、構造化並行を導入し、AI エージェントの実験的記述に必要な安全性と柔軟性の両立を目指した。実装は OCaml、Python、JavaScript、C への複数バックエンドから構成され、研究用インタプリタ、ブラウザ実行、組込み向け C 変換を同じ言語系列で扱える。本論文では、言語設計、型システム、AI 連携、実装方式、サンプルプログラムに基づく評価、ならびに現時点の制限を整理する。

目次

1	はじめに	3
2	設計目標	3
3	言語モデル	3
3.1	アクターとメッセージ	3
3.2	AI 呼び出し	4
3.3	振る舞い変更と動的拡張	4
4	型システム	4
4.1	基本型と構造型	4
4.2	効果注釈	5
4.3	線形型と所有権	5
4.4	構造化並行	5
5	実装	5
5.1	実装構成	5
5.2	ランタイム	6
5.3	パーサと型検査	6
6	評価	6
6.1	記述性	6
6.2	安全性	7
6.3	移植性	7
7	議論	7

7.1	AI エージェントと言語処理系	7
7.2	静的性と動的性の境界	7
7.3	現時点の制限	7
8	関連研究	8
9	おわりに	8

1 はじめに

生成 AI を利用したソフトウェアは、自然言語入力、外部 API 呼び出し、ファイル操作、ネットワーク通信、ユーザ対話、並行タスクを組み合わせて実行する。しかし、多くの実装では、AI 呼び出しは通常の関数呼び出しや HTTP クライアントの薄いラップとして扱われる。そのため、複数の AI エージェントが協調する場合、メッセージの流れ、同期点、権限、データ所有権、失敗時の再試行方針がプログラムの型や構文から見えにくくなる。

本研究の目的は、AI エージェントを明示的なアクターとして記述し、その通信と効果を型付き言語として扱うことである。AIPL は、古典的なアクターモデルを基礎にしながら、`send`、`now`、`future`、`await` による通信、`ai_call` 系の AI 組み込み、型注釈と効果注釈を同じ構文体系に配置する。これにより、AI 呼び出しを含むエージェント群を、単なるスクリプトではなく検査可能な並行プログラムとして表現することを狙う。

本論文の貢献は次の三点である。

- AI エージェントをアクターとして記述する型付き言語 AIPL の設計を示す。
- 型、効果、所有権、線形性、構造化並行を段階的に導入する実装方針を示す。
- OCaml、Python、JavaScript、C の複数処理系を通じて、研究用実行、ブラウザ実行、組込み向け変換を同一言語系列で扱う方法を整理する。

2 設計目標

AIPL の設計目標は以下である。

エージェントの明示化 AI、ユーザ、ツール、ワーカをアクターとして表現し、通信関係を構文上に現す。

同期と非同期の統一 Fire-and-forget の `send`、返信を待つ `now`、Future を返す `future` を使い分けられるようにする。

型による局所的安全性 型注釈された領域では、引数、戻り値、データ構造、所有権を検査する。

AI 効果の分離 ファイル、ネットワーク、AI 呼び出し、状態更新などの効果を注釈し、危険な処理を局所化する。

実験容易性 動的コンパイル、メソッドインジェクション、mock AI provider により、研究段階のエージェント設計を素早く試せるようにする。

複数実行環境 サーバ、ブラウザ、デスクトップ GUI、C 変換、Xinu 風環境を想定し、実装方式の差を比較できるようにする。

3 言語モデル

3.1 アクターとメッセージ

AIPL の基本単位は `class` から生成されるアクターである。各アクターはフィールド、メソッド、メールボックスを持つ。メッセージはメソッド名と引数列から構成され、ランタイムがキューに格納して処理する。

```
class Counter {
  var n = 0;

  method inc() {
    n = n + 1;
    reply(n);
  }
}
```

```
var c = new Counter();
var x = now c.inc();
print(x);
```

`send c.inc()` は送信後に待たない。`now c.inc()` は受信側の `reply` を待つ。`future c.inc()` は `Future` を返し、後続の `await` で結果を取り出す。この三つの通信を明示することで、AI 呼び出しを含む処理の待ち合わせ位置をプログラム上で確認できる。

3.2 AI 呼び出し

AIPL では AI 呼び出しを通常の組み込み関数としてだけでなく、AI アクターとしても扱える。

```
class Reviewer {
  method review(text: string) -> string !{ai} {
    var r = ai_call_with_system(
      "You are a strict reviewer.",
      text
    );
    reply(r);
  }
}
```

Python 版では `AIPL_AI_PROVIDER` により `mock`、`Gemini`、`Anthropic Claude`、`OpenAI` などの `provider` を切り替える設計になっている。`mock provider` を用いると、外部 API やネットワークを使わずにプログラム構造をテストできる。

3.3 振る舞い変更と動的拡張

アクターは `become` により実行中に振る舞いを変更できる。また Python 版では `compile(source)`、`spawn(name, args...)`、`add_method(target, source)` による動的拡張を持つ。これは AI が生成したコード片を制御された形で取り込む実験に有用である。一方で、動的拡張は静的型検査の境界を越えるため、`transient cast` やランタイム検査と組み合わせる必要がある。

4 型システム

4.1 基本型と構造型

Python 版 AIPL は、明示的な型注釈を持つ。基本型は `int`、`float`、`string`、`bool` であり、配列、タプル、レコード、Union 型、型変数、長さ付き配列も扱う。

```
method add(a: int, b: int) -> int {
  return a + b;
}

var xs: array[int] = [1, 2, 3];
var user: record{name: string, age: int}
  = {name: "Alice", age: 30};
```

未注釈領域は `any` として扱われる。したがって AIPL の型システムは、全面的な静的保証よりも、注釈された境界を中心に安全性を強める段階的型付けの立場を取る。

4.2 効果注釈

AI エージェントは、AI 呼び出し、ファイル I/O、ネットワーク、状態更新などを組み合わせる。AIPL ではこれらを Capability 効果として表す。

```
method summarize(path: string) -> string !{fs, ai} {
  var text = read_file(path);
  return ai_call("summarize: " + text);
}
```

効果注釈は、プログラムの安全性だけでなく、実行環境選択にも使える。例えばブラウザのみで実行するプログラムでは fs を禁止し、ネットワークを使うエージェントには net を明示させる、といった制御が可能になる。

4.3 線形型と所有権

AI エージェントでは、API キー、セッション、ファイルハンドル、外部リソースなど、複製や再利用を制限したい値が存在する。Python 版 AIPL では linear T と所有権検査により、move 後の再利用や外部からの非公開フィールド更新を検出する。

```
class Worker {
  pub var name: string = "worker";
  var secret: string = "token";

  method consume(x: linear string) -> string {
    return x;
  }
}
```

pub で公開されたフィールドだけを外部から参照可能にし、内部状態はアクター自身のメソッドを通じて変更する。この制約は、並行実行時の意図しない共有状態更新を減らすためのものである。

4.4 構造化並行

scope { ... } は、スコープ内で生成された並行タスクの寿命を構文ブロックに結びつけるための機能である。エージェントが複数の AI 呼び出しを並列に投げる場合、親スコープの終了時に未完了タスクをどう扱うかが重要になる。構造化並行は、Future の取り忘れや孤立タスクを防ぐための足場となる。

5 実装

5.1 実装構成

AIPL は複数の処理系を持つ。表 1 に主な実装を示す。

表 1: AIPL の主な実装構成

処理系	主なファイル	役割
OCaml 版	src/parser.mly, src/eval_thread.ml, src/repl_thread.ml	アクター実行、型推論、AI 呼び出し、SDL 描画、Web gateway、リモートアクター実験

Python 版	<code>src/python-aipl/grammar.lark,</code> <code>aipl_interp.py, aipl_typeck.py</code>	型注釈、効果、線形型、所有権、動的コンパイル、モデル検査などの研究機能
JavaScript 版	<code>src/browser-abcl/</code>	ブラウザ内実行、Canvas 可視化、サーバ有り/無しのスモークテスト
C 版	<code>src/abcl2c.ml,</code> <code>src/c_translator.ml,</code> <code>src/abcl_gui_runtime.c</code>	POSIX pthread、SDL2 GUI、Xinu 風環境を対象にした C コード生成

OCaml 版は、AIPL/ABCL の基礎的なアクター実行モデルを確認する処理系である。Python 版は、新しい型機能と AI エージェント機能を追加する研究用処理系である。JavaScript 版はブラウザでの可視化と配布容易性を担う。C 版は、生成されたアクター実行を低レベル環境へ持ち込むための実験である。

5.2 ランタイム

ランタイムは、アクターごとのメールボックス、メッセージディスパッチ、返信スロット、Future、組み込み関数表から構成される。同期呼び出し `now` と Future 呼び出し `future` は、内部的にはメッセージ識別子と返信スロットにより関連付けられる。これにより、AI 呼び出しのように遅延が大きい処理も、通常のアクター通信として扱える。

C 版では、`value_t`、`message_t`、`mailbox_t`、`object_t` といったランタイム構造体を生成し、POSIX pthread によって各アクターを実行する。SDL2 GUI ランタイムをリンクすることで、線描画、ボタン、スライダ、哲学者問題やバッファ問題の可視化も可能である。

5.3 パーサと型検査

OCaml 版は `ocamlyacc/ocamllex` 系の構文定義を持ち、Python 版は Lark による文法を持つ。Python 版の型検査器は、クラス、メソッド、関数、変数宣言、式を走査し、注釈された型境界で整合性を検査する。段階的型付けを採用しているため、すべての式を完全に静的解決するのではなく、`any` から具体型へ入る境界をランタイム検査で補う。

6 評価

6.1 記述性

AIPL のサンプルには、カウンタ、Ping-Pong、Bounded Buffer、Dining Philosophers、線分回転、災害復旧可視化、AI サンプルが含まれる。これらは、単純なアクター通信から GUI を伴う並行シミュレーションまでを同じ構文で記述できることを示している。

サンプル領域	確認できる性質
Counter / PingPong	メッセージ送信、返信、基本的なアクター生成
Bounded Buffer	生産者・消費者、同期、キュー状態の可視化
Dining Philosophers	複数アクター間の資源獲得、デッドロック観察
Line Rotation	GUI 描画、周期実行、C/SDL2 変換
AI samples	AI 呼び出し、プロンプト、mock provider によるテスト

6.2 安全性

型注釈、効果注釈、所有権、線形型により、AI エージェント記述で問題になりやすい誤用を局所的に検出できる。例えば、AI 呼び出しを含むメソッドに `!ai` を明示させることで、通常の計算と外部 AI 依存の計算を区別できる。また、`linear` 値の再利用を禁止することで、一度消費した権限トークンや一回限りの資源を再使用するプログラムを検出できる。

一方、AIPL は動的コンパイルやメソッドインジェクションを持つため、完全な静的健全性を主張する処理系ではない。本研究で重視するのは、AI によるプログラム生成や実行時拡張を許容しながら、注釈可能な境界を増やし、危険な領域を明示することである。

6.3 移植性

同じ言語系列を OCaml、Python、JavaScript、C で実装したことにより、次の比較が可能になった。

- OCaml 版では、軽量な型推論とネイティブスレッドを用いた実行を確認できる。
- Python 版では、型システム研究と AI 連携機能の追加を高速に試せる。
- JavaScript 版では、ブラウザ上でサーバ有り/無しの実行形態を検討できる。
- C 版では、pthread、SDL2、Xinu 風環境への展開を検討できる。

これにより、AIPL は単一処理系の言語ではなく、AI エージェント記述のための中間的な設計空間として利用できる。

7 議論

7.1 AI エージェントと言語処理系

AI エージェントは、従来の関数型・オブジェクト指向プログラムとは異なり、外部モデルの非決定的応答を含む。そのため、型システムだけで完全に振る舞いを固定することは難しい。しかし、AI を呼ぶ場所、AI に渡すデータ、AI の結果を受け取る同期点、AI 呼び出しが許可される権限は、言語機能として明示できる。AIPL は、この「AI の非決定性」と「プログラム構造の検査可能性」を分離する。

7.2 静的性と動的性の境界

AIPL は、静的型付けだけでなく、動的コンパイルとメソッドインジェクションを持つ。これは一見矛盾するが、AI エージェント実験では重要である。AI が提案した新しい手続きを実行時に追加し、失敗すれば削除する、あるいは mock provider で検証してから本物の provider に切り替える、といったワークフローを支えるためである。したがって、AIPL の型システムは閉じた世界の完全性ではなく、動的境界を明示して検査することを目標とする。

7.3 現時点の制限

現時点では、処理系間で実装済み機能に差がある。例えば、Python 版の Phase 11 以降の型付き構文は、OCaml 版パーサでは直接扱えない。C 変換では `become` や `select` の一部が制限される。JavaScript 版ではブラウザのセキュリティ制約により、`file://` での直接実行よりもローカルサーバ経由の実行が安定する。これらの差を仕様として整理し、共通中間表現を定義することが今後の課題である。

8 関連研究

AIPL は、アクターモデル、段階的型付け、Capability ベースの効果管理、線形型、構造化並行、AI ツール呼び出しを組み合わせる試みである。Erlang や Akka はアクターによる耐障害並行処理を実用化した。TypeScript や Typed Racket は、動的言語に型境界を導入する段階的型付けの実用例である。Rust は所有権と借用により、共有状態と資源管理を静的に制御する。AIPL はこれらを直接置き換えるものではなく、生成 AI エージェントを記述する小型研究言語として、AI 呼び出しとアクター通信を同じ表現に置く点に特徴がある。

9 おわりに

本論文では、型付き AI エージェント記述言語 AIPL の設計と実装を述べた。AIPL は、アクター、メッセージ、Future、AI 呼び出し、型注釈、効果注釈、所有権、線形型、構造化並行を組み合わせることで、AI エージェントを検査可能な並行プログラムとして記述することを目指す。複数処理系により、研究用機能の追加、ブラウザでの可視化、C による低レベル実行を同じ言語系列で検討できる。

今後は、処理系間の仕様差を縮小し、共通中間表現、形式的意味論、型健全性の範囲、AI 応答の契約記述、実運用向けの権限モデルを整備する。特に、AI が生成したコードを実行時に取り込む場合の型境界と監査可能性は、型付き AI エージェント言語における中心課題である。

参考文献

- [1] Carl Hewitt, Peter Bishop, Richard Steiger. “A Universal Modular ACTOR Formalism for Artificial Intelligence”. Proceedings of IJCAI, 1973.
- [2] Joe Armstrong. “Programming Erlang: Software for a Concurrent World”. Pragmatic Bookshelf, 2007.
- [3] Jeremy G. Siek, Walid Taha. “Gradual Typing for Functional Languages”. Scheme and Functional Programming Workshop, 2006.
- [4] Steve Klabnik, Carol Nichols. “The Rust Programming Language”. No Starch Press, 2018.
- [5] Nathaniel J. Smith. “Notes on structured concurrency, or: Go statement considered harmful”. 2018.